Lesson 3 Teacher's Key

Unreal Engine (optional advanced track)

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Setup adds one new Boolean Input Action (IA_Fire, mapped to Space Bar inside the existing IMC_Default from Lesson 2 - no new Add Mapping Context call needed, since that context is already applied to the player). BP_Bullet's own Event Graph, plus BP_Player's own fire trigger:

```
BP_Bullet's Event Graph:
  Event Tick -> Add Actor World Offset(Self, DeltaLocation = (0,
Speed*DeltaSeconds, 0))
  Event Tick -> Branch(GetActorLocation().Y > 600) -> True: Destroy
Actor(Self)

BP_Player's own Event Graph:
  Enhanced Input Action Event (IA_Fire), Started -> Spawn Actor from
Class(BP_Bullet, Transform = Get Actor Transform(Self))
```

Level 2 - Complete Worked Solution

An Enhanced Input Action Event's Started pin already only fires once per physical press (event-driven, not polled), so Level 1's fire check needs no changes for Level 2. An optional tunable cooldown, if a slower fire rate is wanted:

```
Enhanced Input Action Event (IA_Fire), Started -> Retriggerable Delay
(0.2s) -> Spawn Actor from Class(BP_Bullet, Transform = Get Actor
Transform(Self))
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: play on fire

Add a Play Sound 2D node (using an imported sfx_shoot.wav Sound Wave asset) chained after Spawn Actor from Class:

```
Enhanced Input Action Event (IA_Fire), Started ->
  Spawn Actor from Class(BP_Bullet, Transform = Get Actor Transform(Self))
  Play Sound 2D(sfx_shoot.wav)
```

Visual: give the bullet its own look

BP_Bullet's own sprite (bullet_yellow.png, wired up in M_Bullet_Yellow back in Setup) already makes it read as its own thing rather than a copy of the player Cube, so there's no "give it a

distinct color" gap left to fill the way there is in engines where the bullet still shares the player's look. This idea is a pure size flourish instead: a brief scale pulse in BeginPlay that eases back to normal a moment later.

```
BP_Bullet's Event Graph:
  BeginPlay ->
    Set Actor Relative Scale3D(Self, (1.5, 1.5, 1.5)) ->
    Delay(0.1s) ->
    Set Actor Relative Scale3D(Self, (1.0, 1.0, 1.0))
```

Structure: keep bullet behavior in BP_Bullet

BP_Player should only ever trigger a shot (Spawn Actor from Class + cooldown + sound) - everything about what a bullet actually does once it exists (movement, off-screen cleanup, its own look) belongs inside BP_Bullet's own Event Graph. Keeping this boundary clean is what let the Sound and Visual ideas above each drop into one Blueprint without touching the other.

Complete Level 3 solution (all 3 goals combined)

Everything above combined, across BP_Bullet and BP_Player:

```
BP_Bullet's Event Graph:
  BeginPlay ->
    Set Actor Relative Scale3D(Self, (1.5, 1.5, 1.5)) ->
    Delay(0.1s) ->
    Set Actor Relative Scale3D(Self, (1.0, 1.0, 1.0))
  Event Tick -> Add Actor World Offset(Self, DeltaLocation = (0,
Speed*DeltaSeconds, 0))
  Event Tick -> Branch(GetActorLocation().Y > 600) -> True: Destroy
Actor(Self)

BP_Player's own Event Graph:
  Enhanced Input Action Event (IA_Fire), Started ->
    Retriggerable Delay (0.2s) ->
      Spawn Actor from Class(BP_Bullet, Transform = Get Actor
Transform(Self))
      Play Sound 2D(sfx_shoot.wav)
```

Alternate Path Notes: C++

FireAction is a new UInputAction UPROPERTY bound in the player class's existing SetupPlayerInputComponent(), right alongside the 4 move bindings from Lesson 2 - EnhancedInputComponent->BindAction(FireAction, ETriggerEvent::Started, this, &Class::Fire). No new AddMappingContext() call is needed since IMC_Default is already applied from Lesson 2.

SpawnActor<T>() is a templated function - the template argument (ACppBullet) both creates the instance and gives back a correctly-typed pointer, unlike the Blueprint node's generic Actor return that sometimes needs an explicit cast.

This mirrors the Blueprint path's Spawn Actor from Class node and BP_Bullet's own Event Tick - same underlying engine calls, just from code.

The Level 3 goals map onto C++ calls the same way they map onto Blueprint nodes: UGameplayStatics::PlaySound2D() for sound, and SetActorRelativeScale3D() plus a timer-delayed second call for the scale-pulse visual idea, both called from ACppBullet's own BeginPlay() - keeping the same Structure principle as the Blueprint path.

- Showing both C++ and Blueprint side by side for the same behavior is the clearest way to demystify Blueprints for students who assume "visual" means "not real code" - it's the same engine calls either way.
- This is squarely the optional advanced track - most students should stay on Blueprints; C++ is for students already comfortable with typed languages from Godot, GameMaker, or Unity's C# path.